CLASE 4 - Sistemas Digitales

Semana 4: Temporizadores e interrupciones en el ESP32

1. Conceptos iniciales

- **Sondeo (Polling):** consiste en preguntar repetidamente, mediante un ciclo (`loop`), si ocurrió algún evento. El programa revisa constantemente el estado de un botón, sensor o periférico.
- **Interrupción (Interrupt):** permite que un evento avise al procesador cuando ocurre algo. En lugar de revisar continuamente, el programa puede continuar con otras tareas hasta recibir el aviso. Secuencia básica:
  1. Ocurre un evento.
  2. Se genera una interrupción y se ejecuta la ISR.
  3. Una vez atendida, el programa continúa con su ejecución normal.

Tipos de interrupciones

- **Externas:** se producen por eventos provenientes del exterior del microcontrolador, normalmente mediante un pin GPIO. Por ejemplo, al presionar un botón o cuando un sensor cambia su señal.
- **Internas:** se originan dentro del microcontrolador, por eventos de sus propios periféricos. Por ejemplo, cuando un temporizador alcanza un valor de alarma o cuando otro periférico genera un evento.

ISR (Interrupt Service Routine)

Es la función que se ejecuta cuando ocurre una interrupción. Su finalidad es atender rápidamente el evento que generó el aviso.

- Se ejecuta automáticamente como respuesta a la interrupción.
- No se llama directamente desde el programa principal.
- Debe realizar pocas operaciones y terminar rápidamente.
- Se deben evitar operaciones largas, retardos (`delay()`), esperas y tareas que puedan bloquear la ejecución.

Flanco (Edge)

Es el instante en que una señal digital cambia de un estado lógico a otro:

- **Flanco de subida (Rising):** cambio de 0 a 1.
- **Flanco de bajada (Falling):** cambio de 1 a 0.

Rebote (Debouncing)

Es un fenómeno que ocurre en los botones mecánicos cuando, al presionarlos o soltarlos, sus contactos producen varios cambios eléctricos rápidos antes de estabilizarse.

El microcontrolador puede interpretar esos cambios como varias pulsaciones, aunque el usuario haya presionado el botón una sola vez. Para evitarlo se aplica una técnica de eliminación de rebote, mediante software o circuitos electrónicos.

Registros de la CPU

Son pequeñas posiciones de almacenamiento de alta velocidad que se encuentran dentro del procesador. Permiten guardar temporalmente datos, direcciones e información necesaria para ejecutar instrucciones.

Tipos de memoria

- **Flash:** memoria no volátil donde se almacena el programa y otros datos que deben conservarse cuando se apaga el dispositivo. Tiene mayor capacidad que la SRAM, pero sus operaciones de escritura y borrado son más lentas.
- **SRAM:** memoria volátil de acceso rápido utilizada durante la ejecución del programa. Almacena variables y datos temporales.
- **IRAM (Instruction RAM):** región de memoria RAM destinada a almacenar instrucciones que deben ejecutarse desde RAM. Es especialmente importante para determinadas funciones de interrupción.
- **DRAM (Data RAM):** región de RAM utilizada para almacenar datos y variables durante la ejecución.
- **EEPROM:** memoria no volátil utilizada para conservar pequeñas cantidades de información, incluso sin alimentación. Su disponibilidad y forma de uso dependen del microcontrolador o de la plataforma.

En el ESP32 clásico, la memoria SRAM se organiza en distintas regiones, entre ellas las destinadas a instrucciones (IRAM) y datos (DRAM).

Pull-up

Es una configuración que mantiene una entrada digital en estado HIGH (1) cuando no está conectada a tierra mediante un botón u otro dispositivo.

- **Botón suelto:** la entrada se mantiene en HIGH, aproximadamente 3.3 V.
- **Botón presionado:** la entrada se conecta a GND y pasa a LOW (0 V).

La resistencia pull-up evita que el pin quede flotante cuando el botón está suelto.

---

2. Nivel y flanco

En las señales digitales se pueden identificar dos estados principales:

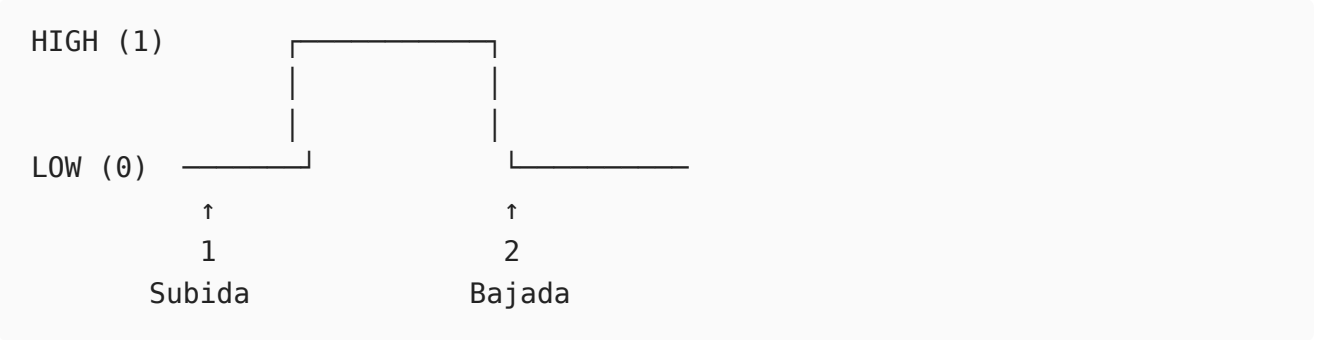
- **LOW (0):** nivel lógico bajo.
- **HIGH (1):** nivel lógico alto.

El flanco es el instante específico en el que la señal cambia de un estado a otro. No representa todo el tiempo que la señal permanece en HIGH o LOW, sino únicamente el momento del cambio.

Flancos de subida y bajada

En una señal digital cuadrada se identifican dos transiciones:

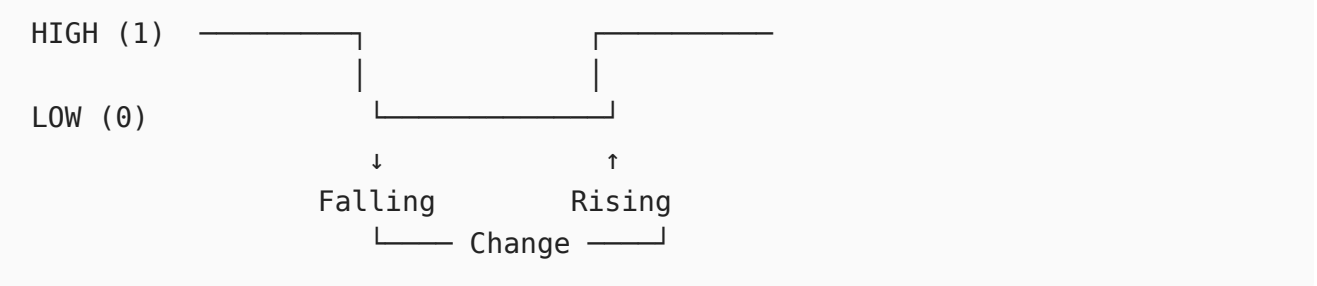
- **1. Flanco de subida:** cuando la señal cambia de LOW (0) a HIGH (1).
- **2. Flanco de bajada:** cuando la señal cambia de HIGH (1) a LOW (0).



Modos de detección de interrupciones

El microcontrolador permite configurar qué cambio de la señal debe generar una interrupción:

1. **Falling (flanco de bajada):** detecta únicamente el cambio de HIGH (1) a LOW (0).
2. **Rising (flanco de subida):** detecta únicamente el cambio de LOW (0) a HIGH (1).
3. **Change (cambio):** detecta ambos cambios, tanto de HIGH a LOW como de LOW a HIGH. Es decir, genera una interrupción cada vez que la señal cambia de estado.

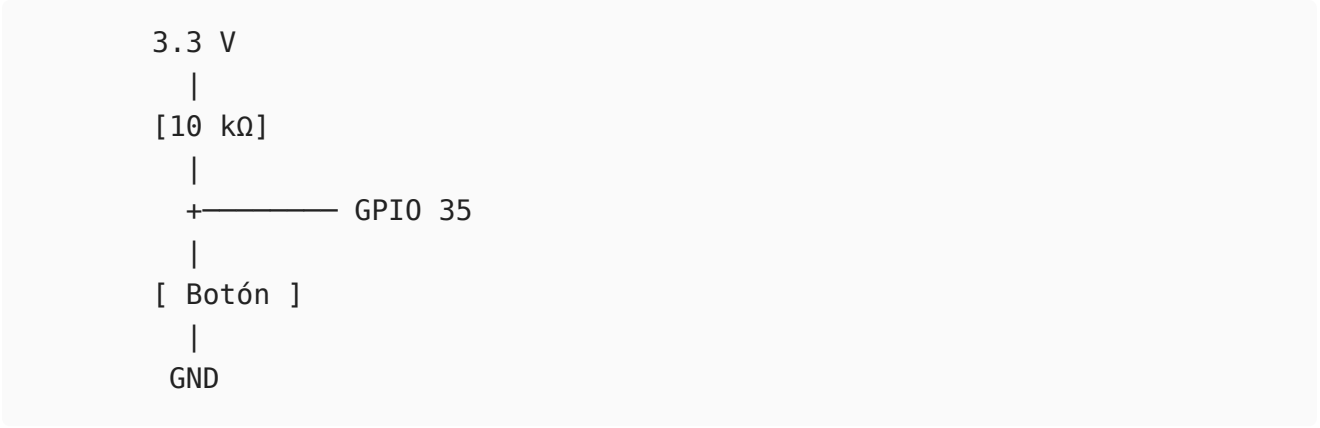


Por ejemplo, si se utiliza un botón con una resistencia pull-up, al presionarlo la señal cambia de HIGH a LOW, por lo que se puede detectar con `FALLING`. Al soltarlo, cambia de LOW a HIGH, por lo que se puede detectar con `RISING`.

3. Pull-up externa

Una resistencia pull-up externa mantiene el pin GPIO en estado HIGH cuando el botón está suelto. Se utiliza cuando el pin no dispone de una resistencia pull-up interna o cuando se necesita una configuración externa.

Ejemplo de conexión:



Funcionamiento:

- **Botón suelto:** la resistencia conecta indirectamente el GPIO a 3.3 V, por lo que se lee HIGH (1).
- **Botón presionado:** el GPIO queda conectado a GND, por lo que se lee LOW (0).

En el ESP32 clásico, GPIO 35 es un pin de solo entrada y no dispone de resistencias pull-up o pull-down internas. Por eso, en este ejemplo se necesita una resistencia externa de aproximadamente 10 kΩ.

---

4. attachInterrupt()

`attachInterrupt()` es una función de Arduino-ESP32 que configura una interrupción para un pin. Permite indicar qué evento debe vigilar el microcontrolador y qué función debe ejecutar cuando ocurra.

Al configurarla se deben establecer tres elementos:

1. **¿Qué pin se va a vigilar?** Se indica el número del GPIO que recibirá la señal.
2. **¿A quién se llama?** Se especifica la función que atenderá la interrupción, es decir, la ISR.
3. **¿Qué cambio debe detectarse?** Se selecciona el modo de interrupción:
   - `FALLING` : cambio de 1 a 0.
   - `RISING` : cambio de 0 a 1.
   - `CHANGE` : cualquiera de los dos cambios.

La configuración se realiza normalmente una sola vez dentro de `setup()`, mediante `attachInterrupt()`.

Hay que distinguir dos configuraciones:

- `**pinMode()**`**:** establece cómo se utilizará el pin, por ejemplo, como entrada (`INPUT`) o entrada con resistencia pull-up interna (`INPUT_PULLUP`).
- `**attachInterrupt()**`**:** configura el evento que generará la interrupción y la función que se ejecutará cuando ocurra.

Ejemplo:

```
const int boton = 35;

void IRAM_ATTR ISR_boton() {
    // Atender el evento del botón
}

void setup() {
    pinMode(boton, INPUT);
    attachInterrupt(digitalPinToInterrupt(boton), ISR_boton, FALLING);
}

void loop() {
    // El programa continúa ejecutando otras tareas
}
```

En este ejemplo, el microcontrolador vigila el GPIO 35 y ejecuta `ISR_boton()` cuando detecta un flanco de bajada. Como GPIO 35 no tiene pull-up interna, el circuito debe contar con una resistencia externa.

---

5. Funcionamiento de la ISR

La ISR es la función encargada de atender una interrupción. No es necesario llamarla desde `loop()` ni desde otra parte del programa: el hardware provoca su ejecución cuando detecta el evento configurado.

Su funcionamiento básico es:

1. Ocurre el evento que genera la interrupción.
2. El hardware avisa al procesador.
3. Se ejecuta la ISR para atender el evento.
4. La ISR termina y el programa continúa con su ejecución normal.

Recomendaciones al escribir una ISR:

- Debe ser breve y ejecutarse rápidamente.
- No debe contener operaciones largas ni retardos como `delay()`.
- Conviene limitarla a registrar que ocurrió el evento o actualizar una variable.
- El procesamiento más complejo debe realizarse fuera de la ISR, por ejemplo, en `loop()`.

---

6. Memoria del programa: Flash, IRAM y DRAM

En el ESP32, el programa y sus datos utilizan distintas regiones de memoria. Es importante distinguir dónde se almacena el código de forma permanente y dónde se ejecuta o se guardan las variables durante el funcionamiento.

Flash

Es la memoria no volátil donde se almacena el programa compilado y otros datos persistentes.

- Conserva la información cuando se apaga el dispositivo.
- Contiene el firmware que se ejecutará al iniciar el microcontrolador.
- El ESP32 clásico utiliza memoria Flash externa al chip principal, integrada normalmente en el módulo.

SRAM: IRAM y DRAM

La SRAM es memoria volátil utilizada durante la ejecución del programa. En el ESP32 se organiza en regiones con diferentes funciones:

- **IRAM (Instruction RAM):** almacena instrucciones de código que deben ejecutarse desde la memoria RAM. Algunas funciones de interrupción se colocan aquí para que puedan ejecutarse incluso cuando no es conveniente acceder a la Flash.
- **DRAM (Data RAM):** almacena variables y datos que utiliza el programa durante su ejecución.

Por tanto, la diferencia principal es que IRAM está destinada a instrucciones y DRAM a datos.

Atributo `IRAM_ATTR`

Es un atributo utilizado en Arduino-ESP32 para indicar que una función debe ubicarse en la IRAM.

Ejemplo:

```
void IRAM_ATTR ISR_boton() {
    // Código breve de la interrupción
}
```

Se utiliza especialmente en determinadas ISR para que su código esté disponible en RAM y pueda ejecutarse durante situaciones en las que el acceso a la Flash no está disponible, como ciertas operaciones de escritura o borrado.

No significa que cualquier función se convierta automáticamente en una interrupción: únicamente indica dónde debe ubicarse su código. Además, las funciones y datos utilizados desde una ISR deben cumplir las restricciones correspondientes para ejecutarse de forma segura desde una interrupción.

Código compilado y ejecución

El proceso general es:

1. Se escribe el programa en el entorno de desarrollo.
2. El compilador transforma el código fuente en instrucciones ejecutables.
3. El programa compilado se almacena en la memoria Flash.
4. Durante la ejecución, el ESP32 utiliza sus mecanismos de memoria y caché para acceder al código, y puede ejecutar determinadas instrucciones desde IRAM.
5. Las variables y datos de ejecución se almacenan en DRAM.

Por eso, no todo el programa tiene que copiarse manualmente a la SRAM: el ESP32 gestiona el acceso al código almacenado en Flash y las regiones de RAM.

Variable `volatile`

`volatile` es un modificador que indica al compilador que una variable puede cambiar por causas externas al flujo normal del programa, por ejemplo, desde una ISR o mediante un periférico.

Evita que el compilador suponga que el valor de la variable permanece constante y que utilice una copia desactualizada al optimizar el código.

Ejemplo:

```
volatile bool botonPresionado = false;

void IRAM_ATTR ISR_boton() {
    botonPresionado = true;
}

void loop() {
    if (botonPresionado) {
        botonPresionado = false;
```

```
        // Procesar el evento fuera de la ISR
    }
}
```

En este ejemplo:

- La ISR cambia `botonPresionado` a `true` cuando ocurre la interrupción.
- El programa principal consulta la variable desde `loop()`.
- Cuando detecta el cambio, realiza el procesamiento fuera de la ISR.

`volatile` no hace que una variable sea automáticamente segura frente a accesos simultáneos ni garantiza operaciones atómicas. Su función principal es evitar ciertas optimizaciones que podrían impedir observar cambios externos.

---

7. Relación entre botón, periférico, bandera y luz

El funcionamiento de un sistema con interrupciones puede entenderse mediante la siguiente secuencia:

1. **Botón:** el usuario lo presiona y cambia el estado eléctrico del pin GPIO.
2. **Periférico GPIO:** detecta el cambio de señal configurado.
3. **Interrupción:** el hardware genera el aviso al procesador.
4. **ISR:** se ejecuta la función que atiende el evento y puede activar una bandera.
5. **Bandera (flag):** es una variable que indica que ocurrió un evento. Por ejemplo, puede cambiar de `false` a `true`.
6. **Programa principal:** consulta la bandera y realiza la tarea correspondiente.
7. **Luz LED:** el programa puede encender o apagar un LED como respuesta a la pulsación del botón.

La idea es que la ISR registre rápidamente el evento y que el programa principal se encargue de realizar las operaciones necesarias, evitando sobrecargar la rutina de interrupción.

---

8. Temporizadores de hardware

Un temporizador de hardware es un periférico que cuenta pulsos de reloj para medir intervalos de tiempo o generar eventos cuando alcanza un valor determinado.

A diferencia de `delay()`, que detiene la ejecución de la tarea que lo utiliza, un temporizador permite medir el tiempo sin tener que detener continuamente el programa.

- `delay()`: pausa la ejecución de la tarea durante un tiempo determinado.
- `millis()`: devuelve el tiempo transcurrido desde el inicio del programa; consultar su valor no bloquea por sí mismo.
- **Hardware Timer:** utiliza un contador interno que avanza mediante pulsos de reloj y puede generar una interrupción al alcanzar una alarma.

Contador del temporizador

El contador es un registro interno del temporizador que aumenta con cada pulso de reloj configurado.

1. La CPU configura el temporizador.
2. El hardware comienza a contar.
3. El contador aumenta según la frecuencia seleccionada.
4. Cuando alcanza el valor de alarma, puede generar un evento o una interrupción.

Reloj y APB

En el ESP32 clásico, el reloj APB utilizado por los temporizadores generales suele trabajar a 80 MHz.

Esto significa que se producen 80 millones de ciclos por segundo.

El tiempo de un ciclo es:

$$T = \frac{1}{f}$$

Para 80 MHz:

$$T = \frac{1}{80,000,000} = 12.5, ns$$

Prescaler

Es un divisor que permite reducir la frecuencia del reloj que utiliza el temporizador.

$$F_{timer} = \frac{F_{reloj}}{N}$$

Donde:

- F_{timer}: frecuencia del temporizador.
- F_{reloj}: frecuencia del reloj de entrada.
- N: divisor o prescaler.

Ejemplo:

$$F_{timer} = \frac{80, MHz}{80} = 1, MHz$$

El temporizador recibe un millón de pulsos por segundo.

Resolución

Es el tiempo que representa cada incremento del contador.

$$R = \frac{1}{F_{timer}}$$

Si el temporizador trabaja a 1 MHz:

$$R = \frac{1}{1,000,000} = 1, \mu s$$

Cada incremento del contador representa 1 microsegundo.

Alarma del temporizador

Es un valor configurado para que el temporizador genere un evento cuando su contador lo alcance.

Las fórmulas principales son:

$$T = Alarma \times R$$

$$Alarma = T \times F_{timer}$$

Donde:

- T: tiempo deseado.
- $Alarma$: cantidad de pulsos que debe contar.
- R: resolución del temporizador.

Autorrecarga (Auto-reload)

Permite que el temporizador vuelva automáticamente a un valor inicial cuando alcanza la alarma, para comenzar un nuevo conteo.

Esto permite generar eventos periódicos sin tener que configurar nuevamente el contador después de cada alarma.

Desbordamiento (Overflow)

Ocurre cuando el contador alcanza su valor máximo y vuelve a cero. Es diferente de alcanzar una alarma: la alarma es un valor configurado, mientras que el overflow depende del límite que puede representar el contador.

El valor máximo de un contador de n bits es:

$$Max = 2^n - 1$$

Ejemplos:

- 8 bits: 255.
- 16 bits: 65 535.
- 64 bits: 18 446 744 073 709 551 615.

En el ESP32 clásico, los temporizadores generales utilizan contadores de 64 bits, por lo que su desbordamiento tarda muchísimo tiempo dependiendo de la frecuencia configurada.

Ejercicio 1

Datos:

- Reloj APB: 80 MHz.
- Frecuencia del temporizador: 1 MHz.
- Tiempo deseado: 0.5 segundos.

Resolución:

$$R = \frac{1}{1,000,000} = 1, \mu s$$

Alarma:

$$Alarma = 0.5 \times 1,000,000 = 500,000$$

Prescaler:

$$N = \frac{80,000,000}{1,000,000} = 80$$

Resultados:

- Prescaler: 80.
- Resolución: 1 microsegundo.
- Alarma: 500 000 pulsos.
- Tiempo: 0.5 segundos.

Ejercicio 2

Datos:

- Reloj APB: 80 MHz.
- Frecuencia del temporizador: 10 kHz.
- Alarma: 2 500 pulsos.

Resolución:

$$R = \frac{1}{10,000} = 0.0001, s = 0.1, ms$$

Tiempo:

$$T = 2,500 \times 0.0001 = 0.25, s$$

Prescaler:

$$N = \frac{80,000,000}{10,000} = 8,000$$

Resultados:

- Prescaler: 8 000.
- Resolución: 0.1 milisegundos.
- Alarma: 2 500 pulsos.
- Tiempo: 0.25 segundos.

Temporizadores del ESP32 clásico

El ESP32 clásico dispone de cuatro temporizadores generales:

- Grupo 0: Timer 0 y Timer 1.
- Grupo 1: Timer 0 y Timer 1.

Estos temporizadores pueden configurarse para medir tiempos, generar alarmas y producir interrupciones.

API

Una API (Application Programming Interface) proporciona funciones predefinidas para utilizar un periférico sin tener que configurar manualmente todos sus registros internos.

En el caso de los temporizadores, permite configurar aspectos como:

- Frecuencia o prescaler.
- Contador.
- Valor de alarma.
- Autorrecarga.
- Función que se ejecutará cuando ocurra el evento.

Interrupciones y temporizadores

Los temporizadores pueden generar interrupciones cuando se cumple una condición, como alcanzar el valor de alarma.

La interrupción avisa al procesador y permite ejecutar una ISR para atender el evento.

Las interrupciones también pueden originarse por otros periféricos, como GPIO, UART o sensores.

Callback

Es una función que el programador proporciona y registra en una API para que el sistema la ejecute cuando ocurra un evento determinado.

Por ejemplo, se puede registrar una función callback para que se ejecute cuando un temporizador alcance su alarma.

Dependiendo de cómo esté implementada la API, el callback puede ejecutarse directamente en una ISR o desde otra tarea. Por eso, es importante conocer el contexto de ejecución y respetar sus restricciones.